Course Name: PCCCS495 – Advanced Programming (OOPS)

Project Title: Layered Banking System using Service Repository Architecture in Java

Student Name: Twinkle Srivastwa

Enrolment Number: 12024052017047

Roll Number: 31

Department: CSE(IOTCSBT)

Year : 2nd Semester: 4

Instructor Name(s):

Dr. Swagatam Basu

Dr. Susovan Jana

Dr. Soumadip Biswas

Semester: Spring 2026

2. Problem Statement

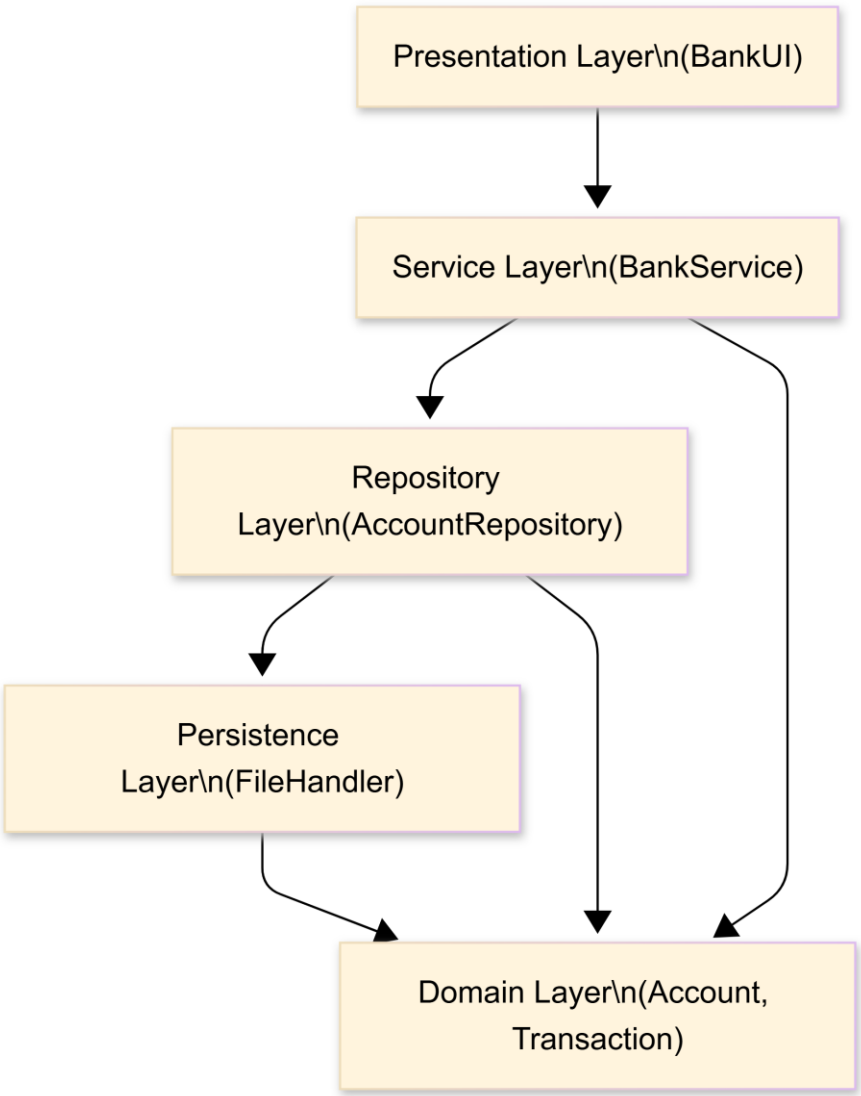
The objective of this project is to design and implement a modular banking system that simulates real-world financial operations such as account creation, deposits, withdrawals, and inter-account transfers. Traditional monolithic implementations often mix business logic with data access, making systems difficult to maintain, extend, and debug.

This project addresses these challenges by adopting a layered architecture combined with the Service–Repository pattern. The system ensures separation of concerns, allowing business logic, data access, and presentation layers to function independently. Additionally, it incorporates robust validation mechanisms such as minimum balance enforcement and overdraft handling.

The system also maintains persistent storage using file-based mechanisms, ensuring that account data is retained across executions. Custom exception handling is implemented to manage invalid operations and maintain system reliability.

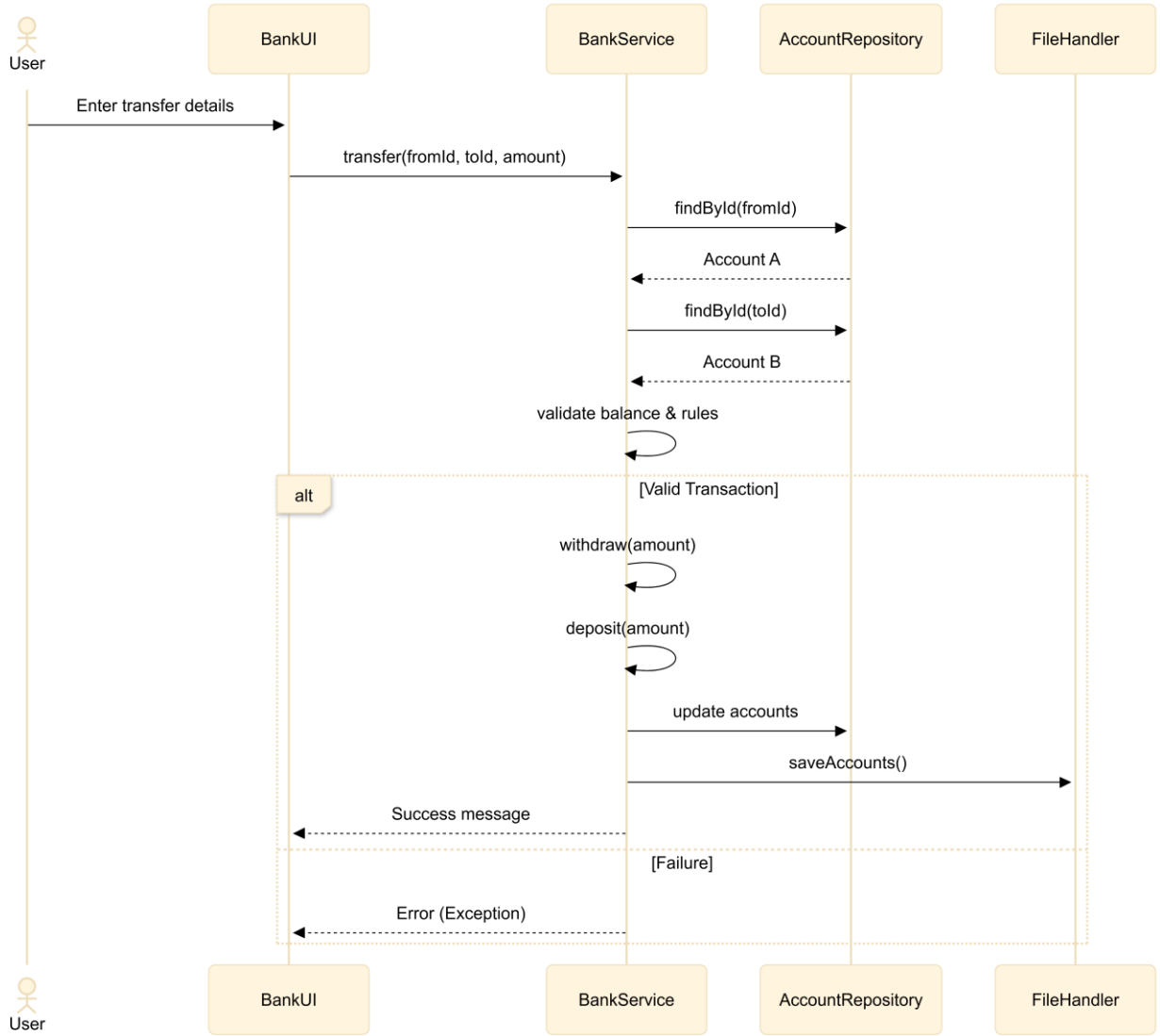
3. System Design

3.1 Component Diagram



3.1 UML Class Diagrams

Sequence Diagram



3.2 Description of Major Classes

Account (Abstract Class):

Defines common attributes such as accountId, accountHolderName, balance, and PIN. It declares abstract methods like withdraw().

SavingsAccount:

Implements withdrawal logic with minimum balance enforcement (₹500).

CurrentAccount:

Supports overdraft functionality with configurable limits.

Transaction:

Stores transaction details including timestamp, type, and involved accounts.

BankService:

Handles core business logic such as deposits, withdrawals, and transfers.

AccountRepository:

Acts as an abstraction layer for data storage and retrieval.

FileHandler:

Manages file-based persistence using CSV format.

3.3 Package Structure

- banking.domain
- banking.service
- banking.repository
- banking.persistence
- banking.exception
- banking.ui

3.4 Interaction Flow

1. User interacts with UI
2. UI calls BankService
3. BankService validates input and processes logic
4. Repository retrieves/stores data
5. FileHandler persists data to file

4. OOP Concepts Demonstrated

4.1 Abstraction

Where Used: Account class

Why Used: To hide implementation details and define a common contract for all account types.

```
public abstract class Account {
   public abstract void withdraw(double amount)
      throws InsufficientBalanceException, InvalidTransactionException;
}
```

Explanation:
The abstract method enforces that all subclasses must implement their own withdrawal logic. This ensures flexibility while keeping a consistent interface across account types.

4.2 Inheritance

Where Used: SavingsAccount and CurrentAccount

Why Used: To reuse common properties and extend behavior for specialized account types.

```
public class SavingsAccount extends Account {
   @Override
   public void withdraw(double amount) {
       // custom logic
   }
}
```

Explanation:
SavingsAccount inherits common attributes like balance and accountId from Account. It overrides the withdraw method to implement specific rules such as minimum balance enforcement.

4.3 Polymorphism

Where Used: Runtime method dispatch in transaction handling

Why Used: To allow a single reference type to handle multiple account types dynamically.

```
Account acc = new SavingsAccount();
acc.withdraw(500);
```

Explanation:
The method call is resolved at runtime, invoking the appropriate implementation based on the actual object type. This enables flexible and scalable code design.

4.4 Encapsulation

Where Used: Private fields in Account class

Why Used: To protect sensitive data and restrict direct access.

```
private double balance;
public double getBalance() { return balance; }
```

Explanation:
Data members like balance and PIN are hidden from direct access. Controlled access through getters ensures data integrity and prevents unauthorized modification.

5. Implementation Highlights

Snippet 1: Deposit Validation

```
public void deposit(double amount) throws InvalidTransactionException {
    if(amount <= 0) throw new InvalidTransactionException("Invalid amount");
```

```
    balance += amount;
}
```

Explanation:

- The method first validates the input amount.
- If the amount is invalid (≤ 0), a custom exception (`InvalidTransactionException`) is thrown.
- This enforces strict business rules at the domain level.
- If valid, the balance is safely updated.

This approach improves **data integrity** and ensures that incorrect values never reach the system state.

Snippet 2: Savings Withdrawal Constraint

```
@Override
public void withdraw(double amount)
    throws InsufficientBalanceException, InvalidTransactionException {
  if (amount <= 0) {
    throw new InvalidTransactionException("Invalid withdrawal amount");
  }
  if (balance - amount < 500) {
    throw new InsufficientBalanceException("Minimum balance of ₹500 must be maintained");
  }
  balance -= amount;

}
```

Explanation:

- First, input validation ensures the withdrawal amount is valid.
- The second condition enforces the **minimum balance rule**.
- If the constraint is violated, a custom exception is thrown.
- Only valid transactions update the balance.

This demonstrates **business rule enforcement through method overriding**, a key aspect of domain-driven design.

Snippet 3: Inter-Account Transfer Logic

```
public void transfer(String fromId, String toId, double amount)
    throws AccountNotFoundException, InsufficientBalanceException, InvalidTransactionException {

  Account from = repository.findById(fromId);
  Account to = repository.findById(toId);

  if (from == null || to == null) {
    throw new AccountNotFoundException("One or both accounts not found");
  }

  from.withdraw(amount);
  to.deposit(amount);

  transactions.add(new Transaction("TRANSFER", amount, fromId, toId));
}
```

Explanation:

- Both source and destination accounts are retrieved from the repository.
- A null check ensures both accounts exist before proceeding.
- Withdrawal is performed first to validate sufficient balance.
- Deposit is then executed on the target account.
- A transaction record is created for audit and tracking purposes.

This design ensures:

- **Consistency** (money is deducted before being added)
- **Traceability** (transactions are logged)

6. Testing & Error Handling

Test Cases

The system was tested using multiple functional scenarios to ensure correctness of banking operations.

Test Case	Input	Expected Output
Create Account (Valid)	Name: "John", Balance: 1000, PIN: 1234	Account created successfully
Create Account (Invalid Name)	Name: "John123"	Error: Only alphabets allowed
Deposit (Valid)	Amount: 500	Balance updated
Deposit (Invalid Amount)	Amount: -100	Error message shown
Withdraw (Savings below min)	Withdraw causing < ₹500	Insufficient balance error
Withdraw (Current overdraft)	Within overdraft limit	Successful withdrawal
Transfer (Valid)	Correct IDs + PIN	Transfer successful
Transfer (Invalid PIN)	Wrong PIN	Access denied
View Balance	Correct ID + PIN	Balance displayed
Admin View	Wrong Admin PIN	Unauthorized access

6.2 Edge Cases Handled

- Empty input fields (name, ID, amount)
- Non-numeric amount input (`NumberFormatException`)
- Negative or zero transaction amounts
- Invalid account IDs
- Incorrect PIN format (not 4 digits)
- Unauthorized admin access

- Empty transaction list

6.3 Error Handling Mechanisms

The system implements both **validation checks** and **exception handling**:

Input Validation (UI Level)

- Name validation using regex (`^[a-zA-Z ]+$`)
- PIN validation (`exactly 4 digits`)
- Amount validation (`> 0`)

Exception Handling (Business Logic)

- `InsufficientBalanceException`
- `InvalidTransactionException`
- `AccountNotFoundException`

Failure Scenarios

- Invalid PIN → transaction blocked
- Insufficient balance → withdrawal denied
- Invalid input → user prompted again
- File errors → handled during save/load

7. Git Workflow

The project followed incremental development with feature-based commits.

Development Progression

The development followed a layered and phased approach:

1. Initialization & Core Setup

- Initialbranch (#1)
- set up initial structure and core classes
- implementation of basic classes

Core domain structure and base classes (`Account`, etc.) were established.

2. Feature Implementation

- added CurrentAccount with overdraft support
- fund transfer functionality
- Exception Handling Structures

Key banking operations and business rules were implemented.

3. Validation & Security

- pin authentication layer addition
- debugging and improved validation
- Error Fix of Pin Authentication

PIN authentication and validation logic were added and refined.

4. Persistence & Architecture

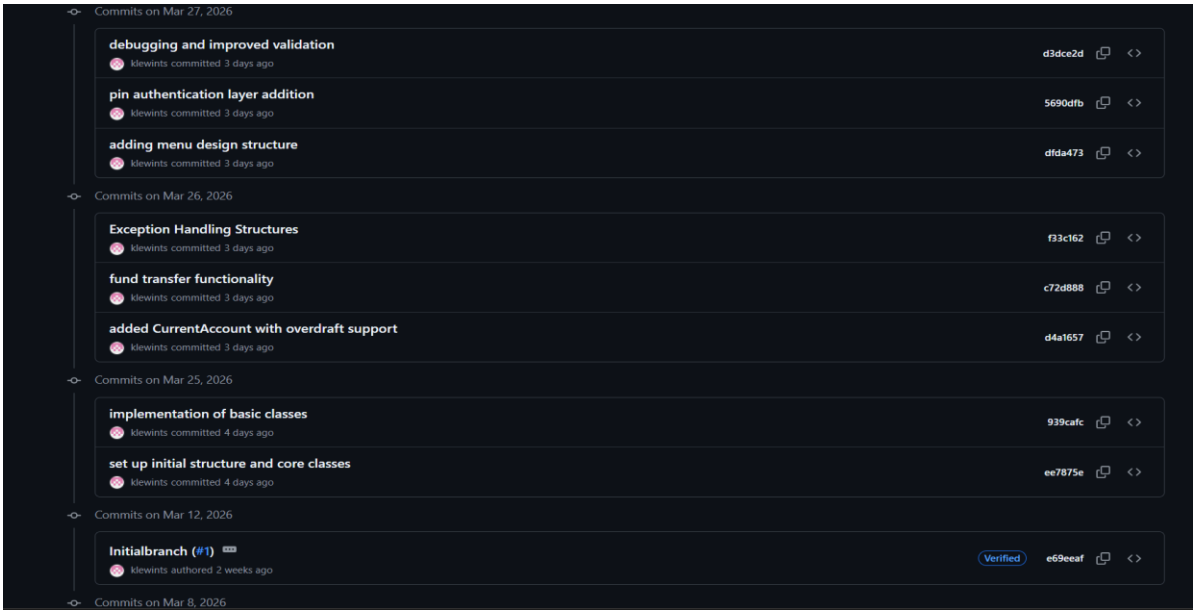
- Adding File Persistence Layer
- pin saving in file persistence
- Repository for account management added

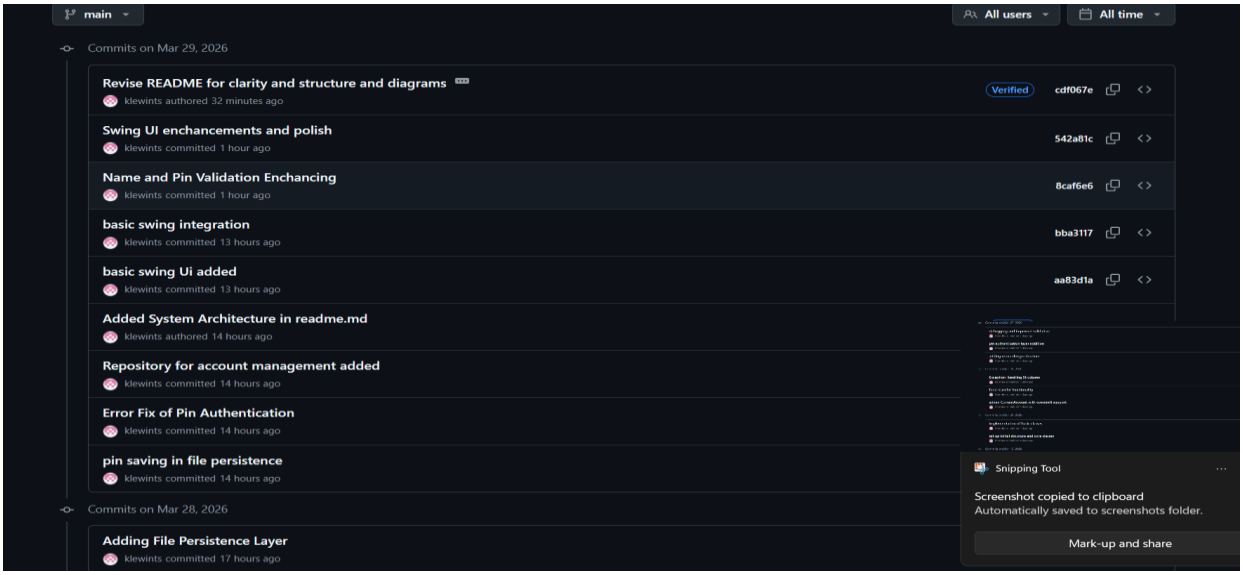
File-based storage and repository abstraction completed the layered architecture.

5. UI Development & Finalization

- basic swing UI added
- basic swing integration
- Swing UI enhancements and polish
- Name and Pin Validation Enhancing
- Revise README for clarity and structure and diagrams

GUI was developed and final improvements were made to validation and documentation.





8. Conclusion & Future Scope

Conclusion

The project successfully demonstrates the implementation of a layered banking system using OOP principles and design patterns. It ensures modularity, maintainability, and scalability.

Future Scope

- Database integration
- Multi-threading support
- REST API development
- Enhanced security features

Appendix

- Initial project proposal (drive link) https://drive.google.com/file/d/1USf56-67OXhwvH3I4p9GsVW8RZZl8K4h/view?usp=sharing
- Review mail (APPROVED_MINOR) https://drive.google.com/file/d/1P0AWgSHYTysmITaK0BIlL0xAeess3gfd/view?usp=sharing